

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Case Study)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO4-AT2-CASE STUDY
Question	Q1 — Personal Finance Management Package in Python (50 Marks)
Section	Section C — Core Implementation (10 Marks)

Q1. Section C — Core Implementation [10 Marks]: Implement the core data model, budget manager, and finance engine designed in Section B.

The complete, runnable implementation of every class and function designed in Section B is given below, exactly as executed to produce the results verified in Sections D and E.

exceptions.py

```
"""
exceptions.py
-----
Custom exceptions for the Personal Finance Management package.
"""

class FinanceError(Exception):
    """Base class for all finance-package errors."""
    pass

class InvalidAmountError(FinanceError):
    """Raised when a transaction amount is not a positive number."""
    def __init__(self, amount):
        self.amount = amount
        super().__init__(f"Transaction amount must be positive, got {amount}")

class InvalidCategoryError(FinanceError):
    """Raised when a transaction category is not a recognised category."""
    def __init__(self, category):
        self.category = category
        super().__init__(f"Unrecognised category '{category}'")

class InvalidDateError(FinanceError):
    """Raised when a transaction date string is not in YYYY-MM-DD format."""
    def __init__(self, date_str):
        self.date_str = date_str
        super().__init__(f"Invalid date '{date_str}', expected YYYY-MM-DD")

class InvalidAccountError(FinanceError):
    """Raised when a transaction references an account that does not exist."""
    def __init__(self, account):
        self.account = account
        super().__init__(f"Unknown account '{account}'")
```

```

class InsufficientFundsError(FinanceError):
    """Raised when an expense would take an account balance below zero."""
    def __init__(self, account, balance, amount):
        self.account = account
        self.balance = balance
        self.amount = amount
        super().__init__(
            f"Insufficient funds in '{account}': balance {balance:.2f}, "
            f"attempted expense {amount:.2f}"
        )

```

models.py

```

"""
models.py
-----

Core data model: Transaction and Account.
"""

from datetime import datetime
from exceptions import InvalidAmountError, InvalidDateError

INCOME_CATEGORIES = {"Salary", "Freelance", "Investment", "Other Income"}
EXPENSE_CATEGORIES = {
    "Food", "Transport", "Utilities", "Rent", "Entertainment",
    "Healthcare", "Shopping", "Education", "Other Expense",
}
ALL_CATEGORIES = INCOME_CATEGORIES | EXPENSE_CATEGORIES
DATE_FORMAT = "%Y-%m-%d"

class Transaction:
    """A single income or expense record."""

    def __init__(self, txn_id, date_str, txn_type, category, amount, description, account):
        try:
            self.date = datetime.strptime(date_str, DATE_FORMAT)
        except ValueError:
            raise InvalidDateError(date_str)

        if amount <= 0:
            raise InvalidAmountError(amount)

        self.txn_id = txn_id
        self.txn_type = txn_type # "Income" or "Expense"
        self.category = category
        self.amount = round(float(amount), 2)
        self.description = description
        self.account = account

    def signed_amount(self):
        """Returns +amount for Income, -amount for Expense (for balance math)."""
        return self.amount if self.txn_type == "Income" else -self.amount

    def to_row(self):
        return [
            self.txn_id, self.date.strftime(DATE_FORMAT), self.txn_type,
            self.category, f"{self.amount:.2f}", self.description, self.account,
        ]

    def __repr__(self):
        return (f"Transaction({self.txn_id}, {self.date.strftime(DATE_FORMAT)}, "
            f"{self.txn_type}, {self.category}, {self.amount:.2f}, {self.account})")

class Account:
    """A named money account (e.g. bank, wallet, cash) with a running balance."""

```

```

def __init__(self, name, opening_balance=0.0):
    self.name = name
    self.opening_balance = round(float(opening_balance), 2)
    self.balance = self.opening_balance

def apply(self, transaction):
    self.balance = round(self.balance + transaction.signed_amount(), 2)

def __repr__(self):
    return f"Account({self.name}, balance={self.balance:.2f})"

```

budget_manager.py

```

"""
budget_manager.py
-----
Loads monthly budget limits (per expense category) from a JSON
file and checks actual spend against those limits.
"""

import json

class BudgetManager:
    def __init__(self, budgets):
        """budgets: dict mapping category -> monthly limit (float)."""
        self.budgets = budgets

    @classmethod
    def from_json(cls, path="budgets.json"):
        with open(path, "r") as f:
            raw = json.load(f)
            return cls({k: float(v) for k, v in raw.items()})

    def limit_for(self, category):
        return self.budgets.get(category)

    def check(self, category, spent):
        """
        Returns a dict describing status for a category's spend against its
        budget limit. If the category has no configured budget, limit is None
        and status is 'Unbudgeted'.
        """
        limit = self.limit_for(category)
        if limit is None:
            return {"category": category, "limit": None, "spent": round(spent, 2),
                    "remaining": None, "status": "Unbudgeted"}
        remaining = round(limit - spent, 2)
        status = "Exceeded" if spent > limit else "Within Budget"
        return {"category": category, "limit": limit, "spent": round(spent, 2),
                "remaining": remaining, "status": status}

```

finance_manager.py

```

"""
finance_manager.py
-----
Core engine: manages accounts and transactions, validates every
input, persists transactions to/from CSV, and exposes balance
lookups used by the reporting layer.
"""

import csv

from models import Transaction, Account, ALL_CATEGORIES
from exceptions import InvalidCategoryError, InvalidAccountError, InsufficientFundsError

```

```

class FinanceManager:
    def __init__(self):
        self.accounts = {}          # name -> Account
        self.transactions = []      # list[Transaction]
        self._next_id = 1

    # ----- account management -----
    def add_account(self, name, opening_balance=0.0):
        self.accounts[name] = Account(name, opening_balance)
        return self.accounts[name]

    def get_balance(self, account_name):
        if account_name not in self.accounts:
            raise InvalidAccountError(account_name)
        return self.accounts[account_name].balance

    # ----- transaction management -----
    def add_transaction(self, date_str, txn_type, category, amount, description, account,
                        allow_overdraft=False):
        """
        Validates and records a transaction. Raises InvalidCategoryError,
        InvalidAccountError, InvalidAmountError, InvalidDateError, or
        InsufficientFundsError as appropriate; on success updates the
        account balance and returns the created Transaction.
        """
        if category not in ALL_CATEGORIES:
            raise InvalidCategoryError(category)
        if account not in self.accounts:
            raise InvalidAccountError(account)

        txn = Transaction(
            txn_id=f"T{self._next_id:04d}", date_str=date_str, txn_type=txn_type,
            category=category, amount=amount, description=description, account=account,
        )

        if txn_type == "Expense" and not allow_overdraft:
            projected = round(self.accounts[account].balance - txn.amount, 2)
            if projected < 0:
                raise InsufficientFundsError(account, self.accounts[account].balance, txn.amount)

        self.accounts[account].apply(txn)
        self.transactions.append(txn)
        self._next_id += 1
        return txn

    # ----- persistence -----
    def save_transactions_csv(self, path="transactions.csv"):
        with open(path, "w", newline="") as f:
            writer = csv.writer(f)
            writer.writerow(["txn_id", "date", "type", "category", "amount", "description", "account"])
            for t in self.transactions:
                writer.writerow(t.to_row())

    def load_transactions_csv(self, path, skip_errors=True):
        """
        Loads transactions from a CSV file, validating and applying each row
        via add_transaction(). Returns (loaded_count, list_of_(row, error)).
        Existing account balances are updated as rows are applied, so
        accounts must already be created before calling this.
        """
        loaded = 0
        errors = []
        with open(path, newline="") as f:
            reader = csv.DictReader(f)
            for row in reader:
                try:
                    amount = float(row["amount"])
                    self.add_transaction(

```

```

        date_str=row["date"], txn_type=row["type"], category=row["category"],
        amount=amount, description=row.get("description", ""),
        account=row["account"], allow_overdraft=True,
    )
    loaded += 1
except Exception as e:
    if skip_errors:
        errors.append((row, e))
        continue
    raise
return loaded, errors

# ----- query helpers used by the reporting layer -----
def transactions_in_month(self, year, month):
    return [t for t in self.transactions if t.date.year == year and t.date.month == month]

```

budgets.json (sample monthly budget config)

```

{
    "Food": 6000,
    "Transport": 3000,
    "Utilities": 4000,
    "Rent": 15000,
    "Entertainment": 2500,
    "Healthcare": 3000,
    "Shopping": 4000,
    "Education": 2000
}

```

transactions_input.csv (sample input data for July 2026)

```

date,type,category,amount,description,account
2026-07-01,Income,Salary,55000,July salary,Bank
2026-07-02,Expense,Rent,15000,July rent,Bank
2026-07-03,Expense,Food,1800,Groceries,Bank
2026-07-05,Expense,Transport,900,Fuel,Bank
2026-07-06,Expense,Utilities,2200,Electricity bill,Bank
2026-07-07,Income,Freelance,8000,Logo design project,Bank
2026-07-08,Expense,Food,1200,Dining out,Cash
2026-07-10,Expense,Entertainment,1500,Movie night,Cash
2026-07-12,Expense,Shopping,4600,New shoes,Bank
2026-07-14,Expense,Healthcare,1200,Pharmacy,Cash
2026-07-15,Expense,Food,2100,Groceries,Bank
2026-07-18,Expense,Transport,1100,Cab rides,Cash
2026-07-20,Expense,Education,1800,Online course,Bank
2026-07-22,Income,Investment,2500,Dividend payout,Bank
2026-07-24,Expense,Utilities,1600,Internet bill,Bank
2026-07-26,Expense,Entertainment,1200,Streaming subscription,Bank
2026-07-28,Expense,Food,1600,Groceries,Bank
2026-07-30,Expense,Shopping,900,Books,Cash

```

main.py (wiring the engine together)

```

"""
main.py
-----
Runs the Personal Finance Management package end to end:
sets up accounts, loads transactions from CSV, generates the
monthly report, and prints account balances.
"""

from finance_manager import FinanceManager
from budget_manager import BudgetManager
from reports import write_monthly_report

manager = FinanceManager()
manager.add_account("Bank", opening_balance=20000)
manager.add_account("Cash", opening_balance=3000)

loaded, errors = manager.load_transactions_csv("transactions_input.csv")

```

```

print(f"Loaded {loaded} transactions, {len(errors)} rejected.")

manager.save_transactions_csv("transactions.csv")

for name, acc in manager.accounts.items():
    print(f"{name} balance: Rs.{acc.balance:.2f} (opening Rs.{acc.opening_balance:.2f})")

budget_mgr = BudgetManager.from_json("budgets.json")
result = write_monthly_report(manager, budget_mgr, 2026, 7, "monthly_report.txt")

print(f"\nSavings rate: {result['savings_rate']}%")
print("Monthly report written to monthly_report.txt")

```

Actual console output of: python3 main.py

```

Loaded 18 transactions, 0 rejected.
Bank balance: Rs.52700.00 (opening Rs.20000.00)
Cash balance: Rs.-2900.00 (opening Rs.3000.00)

Savings rate: 40.92%
Monthly report written to monthly_report.txt

```

Notes on the implementation

- Every Transaction validates its own date and amount inside `__init__`, so an invalid Transaction object can never exist even if a caller bypasses `FinanceManager.add_transaction()` — this is defence in depth, matching the constraint identified in Section A that validation must protect every entry point equally.
- `FinanceManager.add_transaction()` is the single funnel used by both manual transaction entry and CSV import (via `load_transactions_csv`), so category/account/amount/date/funds checks are applied identically regardless of how a transaction enters the system.
- The Cash account intentionally goes negative (Rs.-2900.00) in this run because `load_transactions_csv()` is called with `allow_overdraft=True`, appropriate for importing historical records; interactive entry defaults to `allow_overdraft=False` and raises `InsufficientFundsError` instead, as demonstrated in Section D.
- All monetary values are rounded to 2 decimal places at the point they are set (in Transaction, Account, and `BudgetManager.check`), avoiding floating-point drift across repeated additions.

Conclusion

The implementation directly realises the Section B design: five small, focused modules that together form a working engine. Running `main.py` against the 18-row sample dataset for July 2026 loads every row successfully, correctly updates both account balances, and reports a 40.92% savings rate — all verified against manual calculation in Section E.